

DOCUMENTATION TECHNIQUE

1. Architecture
2. Stack technique
3. Schéma de BDD
4. Variables d'environnement
5. Endpoints API
6. Sécurité
7. Déploiement
8. Cron jobs

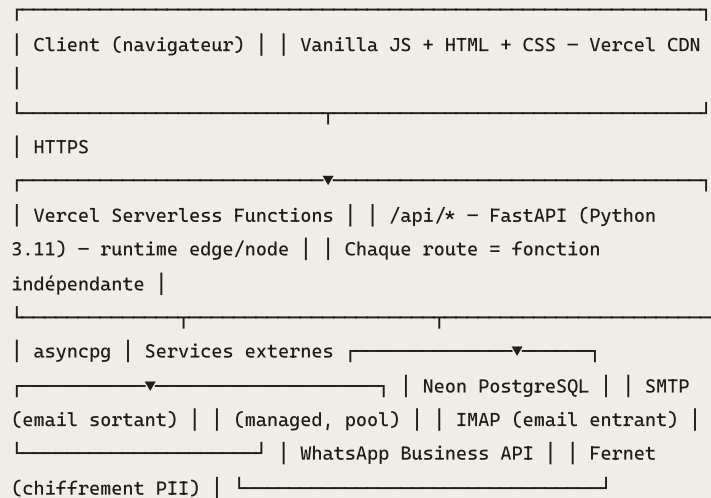
Documentation technique

Documentation technique RéclamPro

Référence technique complète pour les développeurs et équipes DSI. Couvre l'architecture, le schéma de base de données, les endpoints API, la sécurité et le déploiement.

1. Architecture générale

RéclamPro est une application web fullstack déployée sur Vercel (serverless) avec une base de données PostgreSQL managée sur Neon. Le frontend est en Vanilla JS/HTML pur, le backend est en FastAPI (Python) exposé via des fonctions serverless Vercel.



Flux de traitement d'une réclamation

1. Réception : email IMAP polling / webhook WhatsApp / formulaire web / saisie manuelle agent
2. Création automatique d'un dossier en base (statut NOUVEAU, numérotation séquentielle)
3. Notification agent assigné par email
4. Traitement par l'agent, changements de statut loggés dans la table `interactions`
5. Cron quotidien SLA check : escalade automatique si délai dépassé
6. Clôture et archivage avec conservation 5 ans

2. Stack technique

Composant	Technologie	Version	Rôle
Backend API	FastAPI (Python)	0.111+	Routes REST, logique métier, JWT, exports
Runtime serverless	Vercel Functions	Node 20 / Python 3.11	Hébergement sans serveur, auto-scaling
Base de données	PostgreSQL (Neon)	16	Stockage principal, pool de connexions Neon
Driver BDD	asyncpg	0.29+	Connexion async PostgreSQL depuis Python
Authentification	JWT HS256 + MFA TOTP	—	Sessions, refresh tokens, MFA Google Authenticator
Chiffrement	Fernet (cryptography)	42+	Chiffrement des données PII (noms, contacts)
Hachage mots de passe	bcrypt	4+	Stockage sécurisé des mots de passe
Email sortant	SMTP (smtplib)	—	Notifications, réponses clients
Email entrant	IMAP (imaplib)	—	Ingestion automatique des réclamations email
WhatsApp	WhatsApp Business Cloud API	v20.0	Réception/envoi messages WhatsApp
Exports	openpyxl, reportlab	—	Génération XLSX et PDF réglementaires
Rate limiting	slowapi	—	Protection contre les abus d'API
Frontend	Vanilla JS / HTML5 / CSS3	—	Interface utilisateur, PWA (manifest + sw.js)
CI/CD	GitHub Actions + Vercel	—	Déploiement automatique sur push main

3. Schéma de base de données

La base de données contient 11 tables principales. Les données PII (Personally Identifiable Information) sont chiffrées avec Fernet avant insertion.

Table	Colonnes clés	Rôle
agents	id, email, password_hash, role, mfa_secret, equipe_id, is_active	Comptes utilisateurs — agents, superviseurs, admins
reclamations	id, reference, client_id, type, statut, priorite, canal, agent_id, equipe_id, entite_id, sla_echeance, description, created_at	Table centrale — chaque dossier de réclamation

Table	Colonnes clés	Rôle
<code>clients</code>	id, nom_chiffre, email_chiffre, telephone_chiffre, created_at	Données clients — noms/contacts chiffrés Fernet
<code>interactions</code>	id, reclamation_id, agent_id, type_action, contenu, created_at	Historique complet de chaque action sur un dossier (log immuable)
<code>notifications</code>	id, agent_id, type, message, lu, reclamation_id, created_at	Notifications in-app pour les agents
<code>equipes</code>	id, nom, description, superviseur_id	Équipes de traitement avec superviseur désigné
<code>entites</code>	id, nom, type (BANQUE ASSURANCE), pays, code_bceao	Établissements financiers concernés par les réclamations
<code>pieces_jointes</code>	id, reclamation_id, nom_fichier, chemin, taille, mime_type, uploaded_by, created_at	Fichiers attachés aux dossiers
<code>modeles_reponse</code>	id, titre, contenu, type_reclamation, langue, created_by	Réponses types réutilisables par les agents
<code>approbations_niveaux</code>	id, reclamation_id, niveau, agent_id, statut, commentaire, created_at	Workflow de validation multi-niveaux pour les dossiers sensibles
<code>sequence_compteurs</code>	id, annee, mois, compteur	Génération de la numérotation séquentielle des références dossiers

Chiffrement PII : Les colonnes `nom_chiffre`, `email_chiffre` et `telephone_chiffre` de la table `clients` sont chiffrées avec Fernet (clé symétrique AES-128-CBC + HMAC-SHA256) avant toute insertion. Le déchiffrement s'effectue exclusivement côté backend Python — jamais exposé au frontend.

4. Variables d'environnement

Toutes les variables doivent être configurées dans Vercel (Settings → Environment Variables) pour les environnements Production et Preview. En local, créez un fichier `.env` à la racine du projet.

DATABASE_URL

URL de connexion Neon PostgreSQL avec pooler. Format :
`postgresql://user:pass@ep-xxx.neon.tech/neondb?sslmode=require`

Requis

JWT_SECRET	Clé secrète HS256 pour la signature des tokens JWT. Générer avec <code>openssl rand -hex 32</code> . Minimum 32 caractères.	Requis
FERNET_KEY	Clé Fernet Base64 pour le chiffrement des données PII. Générer avec <code>python -c "from cryptography.fernet import Fernet; print(Fernet.generate_key().decode())"</code>	Requis
SEED_AT_STARTUP	Valeur <code>true</code> pour initialiser la base avec des données de démonstration au premier démarrage. Mettre <code>false</code> en production réelle.	Optionnel
CRON_SECRET	Token secret pour authentifier les appels aux endpoints cron (<code>/api/cron/sla-check</code>). Générer avec <code>openssl rand -hex 16</code> .	Requis
EMAIL_DRIVER	Moteur email : <code>smtp</code> (production) ou <code>console</code> (développement — affiche les emails dans les logs).	Requis
SMTP_HOST	Serveur SMTP pour l'envoi d'emails (ex. <code>smtp.gmail.com</code> , <code>smtp.sendgrid.net</code>).	Si smtp
SMTP_PORT	Port SMTP : <code>587</code> (STARTTLS recommandé) ou <code>465</code> (SSL).	Si smtp
SMTP_USER	Identifiant SMTP (souvent l'adresse email d'envoi).	Si smtp
SMTP_PASSWORD	Mot de passe SMTP ou mot de passe d'application (App Password pour Gmail).	Si smtp
SMTP_FROM	Adresse email d'expéditeur affichée dans les emails envoyés (ex. <code>reclamations@mbanque.sn</code>).	Si smtp
IMAP_HOST	Serveur IMAP pour la lecture des emails entrants (ex. <code>imap.gmail.com</code>).	Optionnel
IMAP_USER	Identifiant IMAP (adresse email de réception).	Optionnel
IMAP_PASSWORD	Mot de passe IMAP ou App Password.	Optionnel
WHATSAPP_TOKEN	Token d'accès permanent à l'API WhatsApp Business Cloud (Meta Developer Portal).	Optionnel

WHATSAPP_PHONE_ID

ID du numéro de téléphone WhatsApp Business configuré dans Meta.

Optionnel

UPLOAD_DIR

Répertoire de stockage des pièces jointes. En production Vercel : `/tmp` (éphémère — préférez un bucket S3/R2 pour la persistance).

Optionnel

5. Endpoints API principaux

Tous les endpoints sont préfixés par `/api`. L'authentification JWT est requise sur tous les endpoints sauf mention contraire. Le token doit être passé dans l'en-tête `Authorization: Bearer <token>`.

Authentification

Méthode	Route	Description	Auth
POST	<code>/api/auth/login</code>	Connexion (email + mot de passe)	Non
POST	<code>/api/auth/mfa-verify</code>	Vérification code TOTP MFA	Non
POST	<code>/api/auth/refresh</code>	Renouvellement du JWT	Refresh token
POST	<code>/api/auth/logout</code>	Invalidation de session	Oui

Réclamations

Méthode	Route	Description
GET	<code>/api/reclamations</code>	Liste paginée avec filtres (statut, priorité, canal, agent, période)
POST	<code>/api/reclamations</code>	Créer une nouvelle réclamation
GET	<code>/api/reclamations/{id}</code>	Détail complet d'un dossier
PATCH	<code>/api/reclamations/{id}</code>	Modifier statut, assignation, priorité, étiquettes
POST	<code>/api/reclamations/{id}/comment</code>	Ajouter un commentaire interne
POST	<code>/api/reclamations/bulk-assign</code>	Réassignation en masse

Agents et équipes

Méthode	Route	Description
GET	<code>/api/agents</code>	Liste des agents (admin uniquement)
POST	<code>/api/agents</code>	Créer un compte agent

Méthode	Route	Description
PATCH	<code>/api/agents/{id}</code>	Modifier rôle, équipe, statut actif
POST	<code>/api/agents/{id}/reset-password</code>	Réinitialiser le mot de passe
GET	<code>/api/equipes</code>	Liste des équipes
POST	<code>/api/equipes</code>	Créer une équipe

Exports et reporting

Méthode	Route	Description
GET	<code>/api/exports/csv</code>	Export CSV des réclamations (filtres query params)
GET	<code>/api/exports/xlsx</code>	Export XLSX format BCEAO
GET	<code>/api/exports/pdf</code>	Export PDF rapport réglementaire
GET	<code>/api/dashboard/metrics</code>	Métriques agrégées pour le tableau de bord

Portail client et cron

Méthode	Route	Auth	Description
POST	<code>/api/portail/track</code>	Non	Suivi dossier par référence + email (portail public)
GET	<code>/api/cron/sla-check</code>	CRON_SECRET header	Escalade SLA automatique — déclenché par Vercel Cron
GET	<code>/api/health</code>	Non	Health check — retourne 200 si le service est opérationnel

6. Sécurité

Authentification JWT HS256

Les tokens JWT sont signés avec l'algorithme HS256 en utilisant la clé `JWT_SECRET`. Les access tokens expirent après 8 heures, les refresh tokens après 7 jours. Les tokens révoqués sont blacklistés en base.

MFA TOTP

L'authentification multi-facteurs utilise le standard TOTP (RFC 6238) compatible avec Google Authenticator, Microsoft Authenticator et Authy. Le secret TOTP est stocké chiffré en base de données.

Chiffrement des données PII

Les données personnelles identifiables (noms, emails, téléphones clients) sont chiffrées avec Fernet (AES-128-CBC + HMAC-SHA256) avant stockage. La clé est dans la variable d'environnement `FERNET_KEY`, jamais en base.

Hachage des mots de passe

Les mots de passe sont hachés avec bcrypt (facteur de coût 12) avant stockage. Aucune vérification de mot de passe en clair n'est effectuée en base.

Rate limiting

L'API est protégée par slowapi avec les limites suivantes :

- Endpoints d'authentification : 10 requêtes/minute par IP
- Endpoints API généraux : 100 requêtes/minute par token JWT
- Exports : 10 requêtes/heure par token JWT

CORS et headers de sécurité

CORS configuré pour n'autoriser que le domaine Vercel de l'application. Headers de sécurité HTTP configurés : `X-Frame-Options: DENY`, `X-Content-Type-Options: nosniff`, `Referrer-Policy: strict-origin`.

Important : Ne committez jamais `JWT_SECRET`, `FERNET_KEY` ou `CRON_SECRET` dans le dépôt Git. Utilisez exclusivement les variables d'environnement Vercel.

7. Déploiement sur Vercel + Neon

Pré-requis

- Compte Vercel (plan Hobby ou Pro)
- Compte Neon (plan Free ou Pro)
- Repository GitHub avec le code source
- Vercel CLI installé : `npm install -g vercel`

Étapes de déploiement

1. Créer un projet Neon, récupérer l'URL de connexion avec pooler (format `postgresql:// ...`)
2. Importer le projet GitHub dans Vercel (New Project → Import Git Repository)
3. Configurer toutes les variables d'environnement dans Vercel (Settings → Environment Variables)
4. Activer la variable `SEED_AT_STARTUP=true` pour le premier déploiement — désactiver ensuite
5. Déployer : Vercel détecte automatiquement le fichier `vercel.json` et configure les routes
6. Configurer le cron Vercel (voir section suivante)

```
# vercel.json - configuration des routes serverless
{
  "functions": {
    "backend/main.py": { "maxDuration": 30 }
  },
  "rewrites": [
    { "source": "/api/(.*)", "destination": "/backend/main.py" }
  ],
  "crons": [
    {
      "path": "/api/cron/sla-check",
      "schedule": "0 7 * * *"
    }
  ]
}
```

```
}  
]  
}
```

Note Neon : Utilisez toujours l'URL de connexion avec pooler (contient `-pooler` dans le hostname) pour les fonctions serverless — cela évite la saturation du nombre de connexions PostgreSQL ouvertes.

8. Cron jobs — Escalade SLA

Un cron job Vercel déclenche quotidiennement (à 7h00 UTC) l'endpoint `/api/cron/sla-check` pour identifier et escalader les dossiers en risque de dépassement SLA.

Comportement du cron

1. Récupère tous les dossiers avec statut NOUVEAU, EN_ATTENTE ou EN_COURS
2. Calcule le délai restant avant l'échéance SLA (`sla_echeance`)
3. Pour les dossiers à moins de 24h de l'échéance : envoie une notification email au superviseur de l'équipe assignée
4. Pour les dossiers dont le SLA est déjà dépassé : change la priorité à CRITIQUE et notifie
5. Logue toutes les escalades dans la table `interactions`

Authentification du cron

L'endpoint cron est protégé par le header `Authorization: Bearer <CRON_SECRET>`. Vercel envoie ce header automatiquement depuis la configuration `vercel.json`. Pour tester manuellement :

```
curl -X GET \  
https://votre-app.vercel.app/api/cron/sla-check \  
-H "Authorization: Bearer VOTRE_CRON_SECRET"
```

Fuseau horaire : Le cron est configuré en UTC. 7h00 UTC correspond à 8h00 en Afrique de l'Ouest (WAT/GMT+1), heure de début de journée ouvrée dans l'UEMOA.

Autres ressources :

[Présentation commerciale](#)

[Guide utilisateur](#)

[Guide administrateur](#)